

Phase 2 — KV Cache From Scratch

Goal of this phase: implement the KV cache by hand so I understand it cold. This is the single most-asked LLM-systems interview concept. I built it at two levels: a from-scratch toy attention that shows the mechanism exactly, and a manual decode loop on the real model that proves correctness and measures the real speedup.

1. The concept (plain language)

An autoregressive LLM generates one token at a time. To produce token $t+1$, the attention layer needs the **keys (K)** and **values (V)** of every previous token. Those K and V are deterministic functions of the tokens already generated — they never change once computed.

- **Without a cache:** at every step you re-run the projections for *all* tokens so far. Step t does work proportional to t , so generating n tokens costs $1 + 2 + \dots + n = \mathbf{O(n^2)}$.
- **With a cache:** you store each token's K and V the first time you compute them, and at each new step you only project the **one** new token, then attend over the stored K/V. Each step is **$O(n)$** in attention (one query against n keys), and the projection work per step is **$O(1)$** . Generating n tokens is **$O(n^2)$** total attention but with a far smaller constant, and you eliminate all the redundant projection recomputation.

The cache trades **memory for compute**: you hold K and V for every token in GPU memory so you never recompute them. That stored memory is the central constraint in LLM serving.

Concrete measurement (my toy): K/V projections performed, cached vs uncached —

seq_len	cached	uncached	ratio
16	32	272	8.5×
64	128	4,160	32.5×
128	256	16,512	64.5×

The ratio grows linearly with sequence length — that's the $O(n^2)/O(n)$ gap made visible.

2. What I built

- `engine/kv_cache.py` — from scratch, no HF magic:
- `scaled_dot_product_attention(q, k, v)` — $\text{softmax}(QK^T/\sqrt{d}) \cdot V$ by hand.
- `KVCache` — stores K/V and `append()`s each step's new K/V along the sequence dimension (the literal mechanism).

- `decode_cached / decode_uncached` — identical toy weights, one reuses the cache, one recomputes everything; returns outputs **and** a work counter. Uses float64 so the two paths match to $\sim 1e-12$.
- `kv_cache_memory_bytes(...)` and `model_kv_cache_report(...)` — the memory formula, applied to the real model config.
- `engine/manual_decode.py` — the same idea on the **real Qwen model**:
- `_run_cached` — a manual greedy loop that threads `past_key_values` through each step and feeds **only the new token** next time (with explicit `cache_position` so RoPE positions and cache write slots are correct).
- `_greedy_uncached` — feeds the **entire growing sequence** every step with `use_cache=False`.
- `scripts/kv_cache_demo.py` — prints all three exhibits (toy ratios, real speedup, memory table).
- **Tests (`tests/test_kv_cache.py`)** — toy outputs identical, toy cached does strictly less work, real-model cached and uncached produce **identical tokens**, cached is faster, memory formula is linear in `seq_len`.

Verification (local, MPS): 5/5 phase tests pass. Real model: identical tokens, **~5x speedup** (58.7 tok/s cached vs 11.4 tok/s uncached for 48 tokens).

3. The memory formula (recite this)

KV bytes = $2 \times L \times B \times H_{kv} \times d_{head} \times S \times \text{bytes_per_element}$

- **2** — one cache for keys, one for values
- **L** — number of transformer layers
- **B** — batch size (concurrent sequences)
- **H_{kv}** — number of **key/value** heads (with GQA this is smaller than the number of query heads)
- **d_{head}** — dimension per head
- **S** — sequence length (prompt + generated)
- **bytes** — 2 for fp16/bf16, 1 for int8/fp8

For my model (Qwen2.5-0.5B): $L=24$, $H_{kv}=2$ (GQA!), $d_{head}=64$, fp16 $\rightarrow$ at $S=2048$, $B=1$: **24 MiB**. It scales linearly: $512 \rightarrow 6$ MiB, $2048 \rightarrow 24$ MiB, $8192 \rightarrow 96$ MiB. Now multiply by batch size: 64 concurrent 2048-token sequences = 1.5 GiB **just for KV cache** — which is why memory, not compute, caps how many requests you can batch. This is the motivation for Phase 3 (batching) and vLLM's PagedAttention.

4. Design decisions

Decision	Why
Two levels (toy + real model)	The toy proves I understand the <i>mechanism</i> (explicit K/V append); the real-model loop proves it <i>works and is faster</i> on production weights. Interviewers can probe either.
float64 in the toy	fp32 matmul accumulation order differs between one big projection and per-token projections ($\sim 2e-4$). float64 makes "cached == uncached" exact, so the correctness claim is unambiguous.
Explicit <code>cache_position</code> on the real loop	Calling <code>model.forward</code> directly (not <code>generate()</code>) means I'm responsible for telling the model where the new token sits — for correct RoPE rotation and cache write slot. Without it, cached output diverges from uncached. This bug-and-fix is a great talking point.
Greedy decoding for the correctness test	<code>argmax</code> is deterministic, so cached and uncached must produce <i>byte-identical</i> token sequences — a hard correctness guarantee, not a fuzzy similarity.
Work counter instead of only timing	Timing is noisy; counting K/V projections is a deterministic proof that the cache does asymptotically less work.

5. Interview Q&A

Q: Why is generation without a KV cache $O(n^2)$? A: Each new token attends over all previous tokens, and without a cache you recompute every previous token's K and V at every step. Step t costs $O(t)$; summed over n tokens that's $O(n^2)$. The cache stores K/V once so each step only projects the new token — eliminating the redundant recomputation.

Q: What exactly is stored in the KV cache? A: For every layer and every attention head, the key and value vectors of every token processed so far. Not the queries — queries are only needed for the current token and aren't reused.

Q: Derive the memory cost. A: $2 \text{ (K and V)} \times \text{layers} \times \text{batch} \times \text{kv_heads} \times \text{head_dim} \times \text{seq_len} \times \text{bytes_per_element}$. For Qwen2.5-0.5B in fp16 at 2048 tokens that's $2 \times 24 \times 1 \times 2 \times 64 \times 2048 \times 2 \approx 24$ MiB per sequence. It's **linear** in both sequence length and batch size — the two knobs that blow up serving memory.

Q: What is GQA and how does it affect the cache? A: Grouped-Query Attention uses fewer key/value heads than query heads — multiple query heads share one K/V head. It shrinks the KV cache proportionally (my model has 14 query heads' worth of width but only 2 KV heads), which is a major memory win for long contexts with minimal quality loss. The formula uses H_{kv} , not the query-head count — a common interview gotcha.

Q: The cache makes one request faster. Why does it matter even more for serving many requests?

A: Because KV memory is the binding constraint on batch size. The more you cache per sequence, the fewer sequences fit in memory, so the fewer you can batch, so the lower your throughput. That tension — cache enables fast decode but consumes the memory you need for batching — is exactly what PagedAttention (vLLM) optimizes by allocating the cache in small reusable pages instead of one big contiguous block per sequence.

Q: What breaks if you get `cache_position` / `positions` wrong? **A:** RoPE rotates Q and K by their absolute position. If the cached path tells the model the new token is at position 0 instead of its true position, the rotation is wrong and the output diverges from the uncached reference. I caught this by asserting the two paths produce identical tokens — they only do once positions are threaded correctly.

Q: Prefill vs decode in cache terms? **A:** Prefill processes the whole prompt in one pass and **populates** the cache for all prompt tokens at once (parallel, compute-bound). Decode then **reads and appends** one token at a time (memory-bandwidth-bound). The cache is what makes decode cheap; prefill is what fills it.

6. One-line recall

"I implemented the KV cache from scratch — a toy attention that explicitly appends K/V and shows the $O(n^2) \rightarrow O(n)$ work drop, plus a manual decode loop on the real model that threads past_key_values with correct cache positions, proving identical tokens and $\sim 5\times$ speedup. I can derive its memory cost ($2 \cdot L \cdot B \cdot H_{kv} \cdot d \cdot S$ bytes) and explain why that memory is the binding constraint on batch size."